SMARTBRIDGE
SKILLWALLET
A SmartBridge Product

Code-Layout, Readability and Reusability

Date	12 July 2026
Team ID	
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability, based on a review of the current emotion_detection, ai_response, logging_system, and analytics_dashboard modules.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Consistent 4-space indentation across predictor.py, logger.py, app.py, and the other modules.
2	Proper File Structure	Files and folders are logically organized	Partial	Modules are cleanly separated by responsibility, but duplicate files exist (model.py and bert_model.py both define bert_predict() with different behavior).
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names like predicted_emotion, confidence, and mixed_emotions are self-explanatory.
4	Function / Method Names	Functions are descriptively named	Yes	predict_emotion(), generate_response(), render_dashboard() clearly describe their purpose.
5	Code Comments	Inline and block comments explain logic	Partial	Most modules have docstrings and section headers, but workaround code (e.g., AppLoggerBridge in logger.py) is only loosely explained.
6	Modular Design	Code is split into reusable functions/modules	Yes	Four independent modules (emotion_detection, ai_response, logging_system,

				analytics_dashboard) are orchestrated by app.py.
7	No Redundant Code	Duplicate or unused code is removed	No	logger.py contains two separate logging implementations writing to the same CSV file, and bert_model.py/model.py both define overlapping, conflicting bert_predict() functions.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Try/except blocks wrap most file I/O and logging calls, but one fallback path (AppLoggerBridge's except block) calls the wrong function signature.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
1	predict_emotion()	Python (TensorFlow / PyTorch)	Called from app.py and from compare_models() for side-by-side evaluation	High
2	MockGeminiHandler.generate_response()	Python	Used in app.py as the AI response layer for all five emotion categories	Medium
3	AppLoggerBridge (CSV logger)	Python (pandas, csv)	Used in app.py to persist every interaction	Medium
4	render_dashboard()	Python (Plotly, Streamlit)	Used in app.py to display live analytics	High
5	compare_models()	Python	Available for future evaluation and testing scripts	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear module separation; duplicate mock files should be consolidated.
Readability	4	Descriptive names and docstrings throughout the codebase.

Reusability	3	Core functions are reusable, but overlapping mock implementations reduce clarity.
Documentation / Comments	3	Good high-level docstrings; workaround code needs clearer inline explanation.
Overall Score	3.5	Solid foundation with a few consolidation fixes needed before production use.